

# Project 03 - Flow simulations with Trixi.jl and SPARTA

## Exercise Sheet

Dr. Lambert Theisen & Dr. Georgii Oblapenko

### 1 Overview

In this mini-project you will set up a Couette flow simulation in a channel with a width of 1 cm in Trixi.jl and in SPARTA. Assume a wall temperature of 300 K and wall velocities of  $\pm 1000$  m/s.

First, compare the Trixi.jl and SPARTA solutions in a near-continuum regime, where the Knudsen number is equal to  $Kn = 10^{-3}$ . For larger  $Kn$  values, the Trixi.jl Navier-Stokes simulation will most likely crash. By adjusting the flow density, find the largest Knudsen number for which the continuum Navier-Stokes simulation still works and compare with the SPARTA solution for the same Knudsen number. Most probably, the Trixi.jl solution for a larger Knudsen number will exhibit oscillations. By observing the SPARTA solution, determine possible cause for continuum solution crashes/oscillations (no shocks are present in the flow field!). You can also try running SPARTA at an even higher Knudsen number to see stronger rarefaction effects.

You will need to

- determine the density/number density of the gas corresponding to the given Knudsen number
- determine appropriate number of discretization cells for Trixi.jl (check convergence of velocity profile): simulations might take a while, so approximate convergence is OK
- determine appropriate total simulation time for Trixi.jl for converged solution (see tips below)
- determine appropriate number of discretization cells for SPARTA and timestep based on mean free path resolution and CFL-like conditions (see tips below)
- assess required time-averaging for obtaining smooth results

You will then need to convert results to Paraview, and provide a comparison of the results obtained with the two codes, as well as analysis of the physical effects observed. Specifically, provide plots of

- deviation of  $y$  velocity from linear profile
- temperature

The plots should be 1D (extract from Paraview data via Plot Over Line and export to CSV + your plotting script, or use Paraview Line plotting directly). Use physical units for all axes (Trixi.jl needs scaled units for viscous simulations, see tips section below).

The final submission is a **research-style minipaper** of around **3-4 pages** (excluding figures) together with a **Git repository** containing the Trixi.jl simulation file(s), the SPARTA input files, project documentation, and all metadata needed to reproduce the results.

! Important

**Deadline (end of that date):** 26.07.2026

**Submission:** Via Moodle as repository link and PDF. Add us as members ([@lambert.theisen](#) / [@georgii.oblapenko](#)) to the project repository.

! Important

**Software-engineering requirement.** Treat this as a small research software project, not only as a report. Your work should live in a **Git repository** and be reproducible from the command line. Include code, and a short project documentation.

## 1.1 Learning Outcomes

After completing this project, you should be able to:

- set up Trixi.jl simulations
- set up SPARTA simulations
- use codes operating both in physical (SPARTA) and scaled (Trixi.jl) units
- analyse rarefied gas flow effects

## 2 Couette flow description

In this project, you will be studying a Couette flow: a 1-dimensional flow between two plates (the distance between the plates being  $L = 1$  cm), where the left plate is moving downwards with a  $y$ -velocity of  $-v_0 < 0$  and the right plate is moving upwards with a  $y$ -velocity of  $v_0 > 0$ . Both walls are assumed to have the same temperature  $T_0$ .

For the simplest case of 1) isothermal boundary conditions without slip effects 2) constant viscosity  $\mu$  and thermal conductivity  $\kappa$  one gets a linear velocity profile:

$$v_y(x) = -v_0 + 2\frac{x}{L}v_0.$$

The temperature profile is parabolic:

$$T(x) = T_0 + \frac{2\mu v_0^2}{\kappa} \left[ \frac{x}{L} - \left( \frac{x}{L} \right)^2 \right]$$

A schematic of the flow setup is shown below:

Due to the linear velocity profile, oftentimes, the deviation of the computed  $v_y(x)$  from the linear profile is plotted, to showcase the differences better.

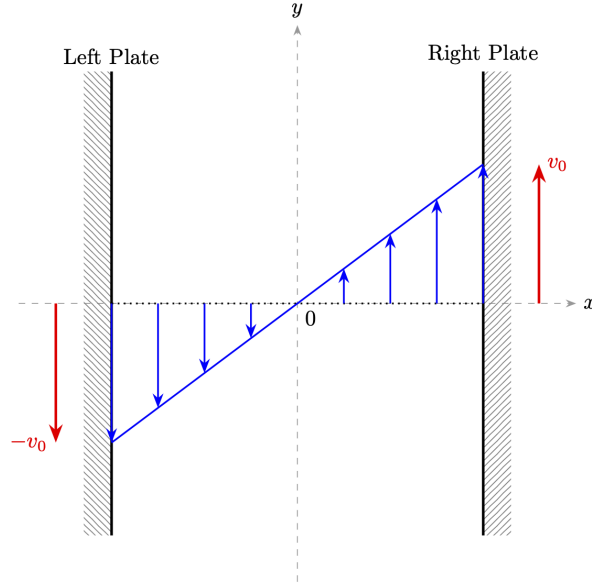


Figure 1: Couette flow schematic

## 2.1 Rarefied flow simulations

We are interested in studying the flow behaviour for more realistic flow regimes, i.e.

- the viscosity and thermal conductivity are temperature-dependent
- the flow might not be continuum

Trixi.jl will not be able to capture the rarefaction effects, but can model the temperature-dependent transport coefficients.

## 2.2 Physical parameters

Consider the following physical parameters:

- model gas is argon with a mass  $m = 6.63 \times 10^{-26}$  kg (data for SPARTA can be found in the SPARTA repository under `data/ar.species`)
- the hard-sphere collision model is used with a diameter  $d = 3.657897 \times 10^{-10}$  m (data for SPARTA can be found in the SPARTA repository under `data/ar.hs`)

The mean-free path for hard spheres is defined as

$$\lambda = \frac{1}{\sqrt{2}n d^2},$$

where  $n$  is the number density of the flow (the usual density  $\rho$  is given by  $\rho = mn$ ).

The viscosity is computed as

$$\mu(T) = \frac{5}{16d^2} \sqrt{\frac{mkT}{\pi}},$$

the thermal conductivity as

$$\kappa = \frac{15k}{4m}\mu.$$

The Prandtl number  $Pr$ , defining the ratio of the momentum diffusivity to thermal diffusivity, is defined as

$$Pr = \frac{c_p \mu}{\kappa}$$

and is equal to  $2/3$  for monatomic gases. Here  $c_p$  is the specific heat capacity at constant pressure, given by  $\frac{5k}{2m}$ .

For computation of the mean free path, you can assume density to be constant throughout the domain (although the Knudsen number does change as the gas flow evolves).

### 3 Trixi.jl set-up tips

Here are some tips for setting up the Trixi simulations, since viscous discretizations were not covered in the lecture/exercise.

#### 3.1 Equations and variable scaling

**Important:** Trixi.jl defines functions `prim2cons(prim, equations)` and `cons2prim(cons, equations)` to convert between conserved variables (the ones we solve for) and primitive variables (that carry more direct meaning and that we use for computation of intermediate quantities, i.e. we need temperature for viscosity). For 2-D flow equations, the first three elements of `prim` are `rho`, `v1`, `v2` (density and x/y velocity components), but the fourth one is dependent on what equation system is used. For `CompressibleEulerEquations2D`, the fourth element of `prim` is the pressure, but for `CompressibleNavierStokesDiffusion2D` it is the temperature. This can be confusing and lead to errors. Moreover, `CompressibleNavierStokesDiffusion2D` assumes that the specific gas constant  $R = k/m$  is equal to 1, so operating in physical variables is impossible.

The solution is to use scaled variables. Install `FlowRef.jl` to compute reference quantities and scale intermediate variables. Don't forget that the domain size will also be scaled with the reference length! `FlowRef.jl` provides several utility functions to compute all the reference quantities from a set of 4 quantities. However, **not all of them** correspond exactly to the implicitly assumed  $R = 1$ .

From gas dynamics:

$$p = nkT = \rho RT.$$

If we use `ref_q = T_v_rho_L(T0, v0, rho, L)` (compute all reference quantities based on reference temperature, velocity, density, length), we get

$$p_{ref} = \rho_{ref} v_{ref}^2, \quad R_{scaled} = \frac{RT_{ref}}{v_{ref}^2} \neq 1.$$

If we however use `ref_q = p_m_rho_L(p, m, rho, L)` ( $m$  is molecular mass), we get

$$v_{ref} = \sqrt{\frac{p_{ref}}{\rho_{ref}}}, \quad R_{scaled} = 1.$$

### 3.2 Type stability

Use `const` for any constant global variables so that the compiler can determine their types. A good way to check type-stability is to run the simulation for not very long and look at the Trixi timing output: if the amount of memory allocated is large (hundreds of megabytes/gigabytes), the code is not type-stable. Fluxes should not allocate! Example of output from a type-stable Trixi.jl setup:

| Trixi.jl             |        | Time         |       |        | Allocations     |      |       |
|----------------------|--------|--------------|-------|--------|-----------------|------|-------|
| Tot / % measured:    |        | 218s / 98.6% |       |        | 29.2MiB / 47.8% |      |       |
| Section              | ncalls | time         | %tot  | avg    | alloc           | %tot | avg   |
| parabolic rhs!       | 471k   | 149s         | 69.5% | 317 s  | 9.00KiB         | 0.1% | 0.02B |
| calculate gradient   | 471k   | 74.5s        | 34.6% | 158 s  | 3.33KiB         | 0.0% | 0.01B |
| volume integral      | 471k   | 29.9s        | 13.9% | 63.5 s | 0.00B           | 0.0% | 0.00B |
| surface integral     | 471k   | 21.6s        | 10.1% | 46.0 s | 0.00B           | 0.0% | 0.00B |
| interface flux       | 471k   | 8.46s        | 3.9%  | 18.0 s | 0.00B           | 0.0% | 0.00B |
| Jacobian             | 471k   | 6.88s        | 3.2%  | 14.6 s | 0.00B           | 0.0% | 0.00B |
| prolong2interfaces   | 471k   | 5.91s        | 2.8%  | 12.6 s | 0.00B           | 0.0% | 0.00B |
| reset gradients      | 471k   | 1.15s        | 0.5%  | 2.43 s | 0.00B           | 0.0% | 0.00B |
| ~calculate gradient~ | 471k   | 391ms        | 0.2%  | 830ns  | 3.33KiB         | 0.0% | 0.01B |
| boundary flux        | 471k   | 68.4ms       | 0.0%  | 145ns  | 0.00B           | 0.0% | 0.00B |

### 3.3 N-S semidiscretization setup

For viscous discretizations in Trixi.jl, one needs to set up both the purely hyperbolic and the parabolic equations:

```
using OrdinaryDiffEqSSPRK
using Trixi
using FlowRef

# set up any constant properties

const v0 = 1000 # wall velocity
const p = T0 * FlowRef.k_B * ndens
const L = 0.01

# use p, m, rho, L as reference quantities
const ref_q = p_m_rho_L(p, m, rho, L)

const gamma = 1.666 # monatomic gas
const prandtl_number = 2.0/3.0
```

```

function mu(u, equations)
    # u is already the vector of primitive variables
    # convert to physical units
    T = u[4] * ref_q.T_ref
    mu = f(T) # TODO
    return mu / ref_q._ref
end

# set up equations
equations = CompressibleEulerEquations2D(gamma)
equations_parabolic = CompressibleNavierStokesDiffusion2D(equations, mu = mu,
                                                            Prandtl = prandtl_number)

```

### 3.4 Grid and initialization

A simple grid setup is via P4est:

```

const coordinates_min = (0.0, 0.0) # minimum coordinates (min(x), min(y))
const coordinates_max = (1.0, 1.0) # maximum coordinates (max(x), max(y)) # 1.0 since L=L_ref

trees_per_dimension = (50, 1) # 1 cell in y direction since flow is periodic
mesh = P4estMesh(trees_per_dimension, polydeg = 1,
                 coordinates_min = coordinates_min, coordinates_max = coordinates_max,
                 periodicity = (false, true))

```

For stability (since walls are fast-moving) and faster convergence it is best to avoid initializing solution with  $v_y = 0$ ; a good approximation is the linear velocity profile:

```

# Set inflow, impose known/expected velocity profile
@inline function initial_condition(x, t, equations)
    v1 = 0.0
    v2 = -v0 + 2 * x[1] * v0

    prim = SVector(rho/ref_q.rho_ref, v1/ref_q.v_ref, v2/ref_q.v_ref, T0/ref_q.T_ref)
    return prim2cons(prim, equations)
end

```

### 3.5 BCs

Boundary conditions need to be prescribed for both the hyperbolic and parabolic parts of the solver:

```

const bc_hyperbolic = (; x_neg = boundary_condition_slip_wall,
                        x_pos = boundary_condition_slip_wall)

### Parabolic boundary conditions ###
const velocity_left = NoSlip((x, t, equations) -> SVector(TODO, TODO))
const heat_bc_wall = Isothermal() do x, t, equations_parabolic
    return temperature(initial_condition(x, t, equations_parabolic),

```

```

                                equations_parabolic)
end
const bc_parabolic_left = BoundaryConditionNavierStokesWall(velocity_left,
                                                            heat_bc_wall)

const velocity_right = NoSlip((x, t, equations) -> SVector(TODO, TODO))
const bc_parabolic_right = BoundaryConditionNavierStokesWall(velocity_right,
                                                            heat_bc_wall)

const bcs_parabolic = (; x_neg = bc_parabolic_left,
                        x_pos = bc_parabolic_right)

```

A special Semidiscretization is required as well:

```

solver = DGSEM(polydeg = polydeg, surface_flux = surface_flux,
              volume_integral = volume_integral)
semi = SemidiscretizationHyperbolicParabolic(mesh, (equations, equations_parabolic),
                                             initial_condition, solver;
                                             boundary_conditions = (bc_hyperbolic,
                                                                    bcs_parabolic))

```

### 3.6 Limiting and CFL-dependent time stepping

Finally, it may be helpful to add some positivity-preserving limiting:

```

save_solution = SaveSolutionCallback(dt = 50.0,
                                     save_initial_solution = true,
                                     save_final_solution = true,
                                     solution_variables = cons2prim)

cfl = 0.75
stepsize_callback = StepsizeCallback(cfl = cfl)

# avoid instabilities
positivity_limiter = PositivityPreservingLimiterZhangShu(thresholds = (1.0e-5, 1.0e-5),
                                                         variables = (pressure,
                                                                    Trixi.density))

callbacks = CallbackSet(summary_callback,
                       analysis_callback,
                       alive_callback,
                       save_solution,
                       stepsize_callback)

#####

sol = solve(ode, SSPRK43(; stage_limiter! = positivity_limiter), adaptive=false;
           dt = 1.0,

```

```
maxiters = 5_000_000,  
ode_default_options(..., callback = callbacks);
```

### 3.7 Number of timesteps

The simulation might take a while to converge. Based on initial conditions, compute the Peclet number:

$$Pe = RePr.$$

This gives the ratio of advection to diffusion; it also indicates the approximate (scaled) time needed for a converged simulation (i.e. how long it takes for diffusive effects to propagate through the domain). For  $Kn = 10^{-3}$ , at least `tspan = (0, 200)` is needed to start observing the impact of the temperature-dependent transport coefficients and seeing more-or-less final density/temperature profiles (probably is still insufficient for true convergence of density/temperature profile). It is useful to set up HDF5 output to (via `SaveSolutionCallback`) to `dt=50.0` to be able to observe convergence in Paraview. One needs to set `maxiters` in the `solve` routine to a larger value, since it might default to 100\_000 or 1\_000\_000, meaning the simulation will run for only 100k/1e6 timesteps, which might not be sufficient for convergence.

### 3.8 Output

By default, Trixi will output scaled primitive variables, i.e. pressure, density, and the two velocity components. You can convert them to quantities of interest directly in Paraview, or write your own output routine (see `Trixi.jl` exercise).

## 4 SPARTA tips

You can find the relevant data for argon in `data/ar.hs` (collision data for the hard spheres model) and `data/ar.species` (argon molecular data).

Since the flow is 1-dimensional, use 1 cell in the y direction and periodic boundary conditions.

Aim for 30-50 particles per cell at the start of the simulation (use less if simulation cost is too high), and cell size not larger than the mean free path.

Based on the cell size and the wall velocity, estimate a CFL-like condition for the timestep.

Use multiple `fix <fix_name> ave/grid ...` commands to perform multiple averagings of the computed data at various timesteps, and use multiple `dump` commands to output these averaged data at different timesteps. This should allow you to assess whether the flow has reached steady state. A total simulation time of 6-8ms is probably required to reach steady state for  $Kn = 10^{-3}$ .



## 5 Check for solution physicality

From constant-valued transport coefficient equation, one can derive an expression for the change in temperature in the middle of the channel ( $x = L/2$ ) relative to the wall temperature:

$$\Delta T = \frac{\gamma - 1}{2} Pr M^2 T_w,$$

where the Mach number  $M$  is computed based on the wall temperature and wall velocity  $v_0$ . You can use this to assess whether the temperature profile you obtained is physically reasonable.